DESK RACER: MARIO KART LAN & APK BLUEPRINT

COMPLETE GUIDE

Complete Chat Archive: Architecture, Hotspot Multiplayer, 3D Engine Code & Cloud APK Builder

User Prompt 1: *"Create a desk racer mobile apk like mario kart playable using same network like friend use my hotspot we play together"*
Overview: Building a "Desk Racer" or Mario Kart-style arcade game that runs over a local mobile hotspot (zero internet required) can be done in two ways: **building a fast web-based 3D game wrapped into an APK using Capacitor**, or **using a native game engine like Unity or Godot**.

PART 1 Option 1: The Fast-Track Web-to-APK Stack (React + Three.js + Capacitor)

For a local hotspot multiplayer game, a standard WebRTC or WebSocket server needs to run *directly on the Host's phone* or via peer-to-peer local discovery.

Component	Technology	Role
3D Engine & Physics	**React Three Fiber (R3F)** + `@react-three/rapier`	Renders the desk environment (books, coffee mugs, rulers as ramps) and arcade kart physics.
Local Hotspot Networking	**Capacitor Node.js / Local WebSockets** or **Wi-Fi Direct plugin**	Allows Phone A (the Hotspot Host) to run a lightweight local socket server on its gateway IP (usually `192.168.43.1` or `192.168.137.1` on Android), while Phone B connects directly to that IP.
APK Wrapper	**Vite + Capacitor**	Compiles your React + Three.js bundle into a native Android `.apk` with full touchscreen and gyroscope access.

1. Arcade Kart Physics Trick (The "Mario Kart" Feel): Never use realistic 4-wheel raycast suspension for a Mario Kart or Micro Machines-style desk racer—it flips too easily and feels sluggish. Instead, use the **Rolling Sphere trick**:

- The physics collider is actually a **single invisible sphere** (`RigidBody` in Rapier).
- You apply forward/backward forces to the sphere, and rotate the **3D Kart Mesh** separately to match the sphere's position while adding visual tilt and drift angles when turning.

2. Complete Prototype Setup & Local Multiplayer Server

Terminal — Install Dependencies — bash

```
npm create vite@latest desk-racer -- --template react
cd desk-racer
npm install three @react-three/fiber @react-three/drei @react-three/rapier socket.io-client
```

Multiplayer State Server (server.js - Run on Host or Termux/Local Node) — javascript

```
import { Server } from "socket.io";

const io = new Server(4000, { cors: { origin: "*" } });
const players = {};

io.on("connection", (socket) => {
  console.log("Racer joined:", socket.id);
  // Spawn player with a random color and starting offset
  players[socket.id] = {
    id: socket.id,
    x: Math.random() * 4 - 2, y: 1, z: 0, rotY: 0,
    color: "#" + Math.floor(Math.random()*16777215).toString(16),
    item: null
```

PART 1 (CONT.) React Three Fiber Kart Controller & Desk Arena (src/App.jsx)

This component sets up the physics sphere controller, third-person camera lerp, remote player interpolation, and the 3D wooden desk track with book obstacles and a yellow ruler jump ramp.

src/App.jsx — Part A: Imports, LocalKart Physics & RemoteKarts — jsx

```
import React, { useRef, useState, useEffect } from "react";
import { Canvas, useFrame } from "@react-three/fiber";
import { Physics, RigidBody, CuboidCollider } from "@react-three/rapier";
import { Environment, PerspectiveCamera, Text } from "@react-three/drei";
import { io } from "socket.io-client";
import * as THREE from "three";

// Connect to the Hotspot Host IP (Replace with dynamic input in UI)
let socket;

function LocalKart({ controls, socket }) {
  const bodyRef = useRef();
  const meshRef = useRef();
  const [rotationY, setRotationY] = useState(0);

  useFrame((state, delta) => {
    if (!bodyRef.current || !meshRef.current) return;
    const pos = bodyRef.current.translation();

    // Steering
    let newRotY = rotationY;
    if (controls.left) newRotY += 3.0 * delta;
    if (controls.right) newRotY -= 3.0 * delta;
    setRotationY(newRotY);

    // Forward vector based on current steering angle
    const forward = new THREE.Vector3(0, 0, -1).applyAxisAngle(new THREE.Vector3(0, 1, 0), newRotY);

    // Apply arcade impulse
    const speed = 25 * delta;
    if (controls.gas) {
      bodyRef.current.applyImpulse({ x: forward.x * speed, y: 0, z: forward.z * speed }, true);
    }
    if (controls.brake) {
      bodyRef.current.applyImpulse({ x: -forward.x * (speed * 0.6), y: 0, z: -forward.z * (speed * 0.6) },
true);
    }

    // Sync visual kart mesh to the physics sphere
    meshRef.current.position.set(pos.x, pos.y - 0.4, pos.z);
    meshRef.current.rotation.y = newRotY;

    // Camera follow
    const camOffset = new THREE.Vector3(0, 3, 6).applyAxisAngle(new THREE.Vector3(0, 1, 0), newRotY);
    state.camera.position.lerp(
      new THREE.Vector3(pos.x + camOffset.x, pos.y + camOffset.y, pos.z + camOffset.z),
      0.1
    );
    state.camera.lookAt(pos.x, pos.y + 0.5, pos.z);

    // Emit position to hotspot peers
    if (socket && socket.connected) {
      socket.emit("move", { x: pos.x, y: pos.y - 0.4, z: pos.z, rotY: newRotY });
    }
  });

  return (
    <>
      <RigidBody ref={bodyRef} colliders="ball" mass={2} linearDamping={1.5} angularDamping={2.0}
position={[0, 2, 0]}>
        <mesh visible={false}>
          <sphereGeometry args={[0.5, 16, 16]} />
        </mesh>
      </RigidBody>
      <group ref={meshRef}>
        {/* Kart Chassis */}
        <mesh castShadow>
          <boxGeometry args={[1, 0.5, 1.8]} />
          <meshStandardMaterial color="#ff3366" />
```

src/App.jsx — Part B: DeskArena, Hotspot Lobby & Mobile Touch Controls jsx

```jsx
function DeskArena() {
  return (
    <RigidBody type="fixed" friction={1}>
      <mesh receiveShadow position={[0, 0, 0]}>
        <boxGeometry args={[60, 1, 60]} /><meshStandardMaterial color="#8b5a2b" />
      </mesh>
      <mesh castShadow receiveShadow position={[5, 1, -10]}>
        <boxGeometry args={[6, 1.5, 8]} /><meshStandardMaterial color="#2c3e50" />
      </mesh>
      <mesh castShadow receiveShadow position={[-5, 0.8, -8]} rotation={[0.25, 0, 0]}>
        <boxGeometry args={[4, 0.2, 10]} /><meshStandardMaterial color="#f1c40f" />
      </mesh>
    </RigidBody>
  );
}

export default function App() {
  const [hostIp, setHostIp] = useState("192.168.43.1:4000");
  const [connected, setConnected] = useState(false);
  const [remotePlayers, setRemotePlayers] = useState({});
  const [controls, setControls] = useState({ left: false, right: false, gas: false, brake: false });

  const connectToHotspot = () => {
    socket = io(`http://${hostIp}`);
    socket.on("connect", () => setConnected(true));
    socket.on("init", ({ id, players }) => {
      const others = { ...players }; delete others[id]; setRemotePlayers(others);
    });
    socket.on("playerJoined", (p) => setRemotePlayers((prev) => ({ ...prev, [p.id]: p })));
    socket.on("playerMoved", (p) => setRemotePlayers((prev) => ({ ...prev, [p.id]: p })));
    socket.on("playerLeft", (id) => {
      setRemotePlayers((prev) => { const copy = { ...prev }; delete copy[id]; return copy; });
    });
  };

  return (
    <div style={{ width: "100vw", height: "100vh", position: "relative", userSelect: "none", overflow:
"hidden" }}>
      {!connected && (
        <div style={{ position: "absolute", zIndex: 10, top: 20, left: 20, background: "rgba(0,0,0,0.8)",
padding: 16, borderRadius: 8, color: "white" }}>
          <h3>Desk Racer LAN Lobby</h3>
          <input value={hostIp} onChange={(e) => setHostIp(e.target.value)} placeholder="Host IP:Port"
style={{ padding: 8, marginRight: 8 }} />
          <button onClick={connectToHotspot} style={{ padding: "8px 16px" }}>Join Hotspot Race</button>
        </div>
      )}
      <Canvas shadows>
        <PerspectiveCamera makeDefault position={[0, 5, 10]} />
        <ambientLight intensity={0.6} /><directionalLight castShadow position={[10, 20, 10]} intensity=
{1.2} />
        <Physics>
          <LocalKart controls={controls} socket={socket} />
          <RemoteKarts remotePlayers={remotePlayers} />
          <DeskArena />
        </Physics>
      </Canvas>
      {/* Touch Overlay Buttons for Left/Right Steering and Drift/Gas omitted for brevity; full
implementation on Page 5 */}
    </div>
  );
}
```

PART 2 Option 2: Zero-Config Native APK Setup (Godot 4 or Unity)

- **Engine Selection: Godot 4** (Lightweight, exports an Android APK in 30 seconds) or **Unity (Netcode for GameObjects + Unity Transport)**.
- **How Hotspot Networking Works Under the Hood:** When Phone A turns on its Android Wi-Fi Hotspot, Android acts as a local router (`192.168.43.1`). Using **UDP Broadcast Auto-Discovery**, the Host APK broadcasts `"DESK_RACER_HOST_HERE"` to `255.255.255.255:8888` every second. Your friend's phone listens on port 8888, grabs the sender's IP automatically, and connects on port 7777.

User Prompt 2: *"Bro u were giving me the apk please"*
Solution: Below is the **1-Command GitHub Cloud APK Builder** (which compiles the game into a downloadable `DeskRacer-debug.apk` on GitHub's servers without needing Android Studio) and the **Single-File Complete 3D Game (`index.html`)** with Mario Kart hop-drifting, item boxes, bouncy marbles, and P2P multiplayer.

PART 4 Complete Single-File 3D Game (index.html — Part 1 of 2)

index.html — Part 1: HTML UI, Touch Controls, PeerJS Multiplayer & 3D Desk Track `html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0, user-
scalable=no" />
  <title>Desk Kart LAN</title>
  <style>
    * { box-sizing: border-box; user-select: none; -webkit-user-select: none; margin: 0; padding: 0; }
    body, html { width: 100vw; height: 100vh; overflow: hidden; background: #111; font-family: system-ui,
sans-serif; }
    #ui { position: absolute; inset: 0; pointer-events: none; z-index: 10; }
    .panel {
      pointer-events: auto; position: absolute; top: 12px; left: 12px;
      background: rgba(15, 23, 42, 0.88); color: #fff; padding: 12px 16px;
      border-radius: 12px; border: 1px solid rgba(255,255,255,0.15);
      display: flex; gap: 8px; align-items: center; flex-wrap: wrap; max-width: 92vw;
    }
    .panel input {
      padding: 8px 10px; border-radius: 6px; border: 1px solid #475569;
      background: #1e293b; color: #fff; font-weight: bold; width: 110px; text-transform: uppercase;
    }
    .btn { pointer-events: auto; padding: 8px 14px; border-radius: 8px; border: none; font-weight: 700;
cursor: pointer; background: #3b82f6; color: white; }
    .btn.host { background: #10b981; }
    #hud { position: absolute; top: 12px; right: 16px; color: #fff; font-size: 20px; font-weight: 900;
text-shadow: 0 2px 4px #000; text-align: right; }
    .ctrl-left { position: absolute; bottom: 24px; left: 24px; display: flex; gap: 16px; pointer-events:
auto; }
    .ctrl-right { position: absolute; bottom: 24px; right: 24px; display: flex; gap: 14px; align-items:
flex-end; pointer-events: auto; }
    .touch-btn {
      width: 76px; height: 76px; border-radius: 50%; border: 3px solid rgba(255,255,255,0.4);
      background: rgba(30, 41, 59, 0.75); color: white; font-size: 26px; font-weight: bold;
      display: flex; align-items: center; justify-content: center; backdrop-filter: blur(4px);
    }
    .touch-btn:active { transform: scale(0.92); background: rgba(255,255,255,0.3); }
    .btn-gas { width: 88px; height: 88px; background: rgba(34, 197, 94, 0.85); font-size: 18px; }
    .btn-drift { width: 72px; height: 72px; background: rgba(245, 158, 11, 0.85); font-size: 15px; }
    .btn-item { width: 72px; height: 72px; background: rgba(239, 68, 68, 0.85); font-size: 15px; }
  </style>
  <script src="https://cdnjs.cloudflare.com/ajax/libs/three.js/r128/three.min.js"></script>
  <script src="https://unpkg.com/peerjs@1.5.2/dist/peerjs.min.js"></script>
</head>
<body>
  <div id="ui">
    <div class="panel" id="lobby">
      <span id="status">Room Code:</span>
      <input id="roomInput" placeholder="E.G. KART1" maxlength="8" />
      <button class="btn host" onclick="hostGame()">HOST</button>
      <button class="btn" onclick="joinGame()">JOIN</button>
    </div>
    <div id="hud">
      <div id="itemDisplay">ITEM: NONE</div>
      <div id="speedDisplay" style="font-size:14px; color:#38bdf8;">DRIFT BOOST READY</div>
    </div>
    <div class="ctrl-left">
      <div class="touch-btn" id="btnLeft">◀</div>
      <div class="touch-btn" id="btnRight">▶</div>
    </div>
    <div class="ctrl-right">
      <div class="touch-btn btn-item" id="btnItem">FIRE</div>
      <div class="touch-btn btn-drift" id="btnDrift">DRIFT</div>
      <div class="touch-btn btn-gas" id="btnGas">GAS</div>
```

PART 4 (CONT.) Complete Single-File 3D Game (index.html — Part 2 of 2)

index.html — Part 2: 3D Scene, Karts, Drift Boost, Item Roulette & Physics Loop — javascript

```
    const scene = new THREE.Scene();
    scene.background = new THREE.Color(0x1e293b);
    scene.fog = new THREE.Fog(0x1e293b, 35, 95);
    const camera = new THREE.PerspectiveCamera(65, window.innerWidth / window.innerHeight, 0.1, 200);
    const renderer = new THREE.WebGLRenderer({ antialias: true });
    renderer.setSize(window.innerWidth, window.innerHeight);
    renderer.shadowMap.enabled = true;
    document.body.appendChild(renderer.domElement);

    scene.add(new THREE.AmbientLight(0xffffff, 0.7));
    const dirLight = new THREE.DirectionalLight(0xfffbeb, 1.1);
    dirLight.position.set(25, 45, 25); dirLight.castShadow = true;
    scene.add(dirLight);

    const desk = new THREE.Mesh(new THREE.BoxGeometry(80, 2, 80), new THREE.MeshStandardMaterial({ color:
0x8b5a2b, roughness: 0.6 }));
    desk.position.y = -1; desk.receiveShadow = true; scene.add(desk);

    const obstacles = [];
    function addBook(x, z, w, h, d, color) {
      const mesh = new THREE.Mesh(new THREE.BoxGeometry(w, h, d), new THREE.MeshStandardMaterial({ color
}));
      mesh.position.set(x, h / 2, z); mesh.castShadow = true; mesh.receiveShadow = true;
      scene.add(mesh); obstacles.push({ x, z, rx: w / 2 + 0.8, rz: d / 2 + 0.8 });
    }
    addBook(0, -15, 16, 2, 12, 0x2563eb);
    addBook(14, 5, 8, 2.5, 14, 0xdc2626);
    addBook(-16, 8, 10, 2, 10, 0x16a34a);

    const ramp = new THREE.Mesh(new THREE.BoxGeometry(6, 0.3, 12), new THREE.MeshStandardMaterial({ color:
0xfacc15 }));
    ramp.position.set(-8, 0.9, -12); ramp.rotation.x = 0.22; scene.add(ramp);

    const itemBoxes = [];
    for (let i = 0; i < 6; i++) {
      const box = new THREE.Mesh(new THREE.OctahedronGeometry(0.9), new THREE.MeshStandardMaterial({
color: 0x38bdf8, transparent: true, opacity: 0.85 }));
      const angle = (i / 6) * Math.PI * 2;
      box.position.set(Math.cos(angle) * 20, 1.2, Math.sin(angle) * 20);
      scene.add(box); itemBoxes.push(box);
    }

    function createKart(color) {
      const group = new THREE.Group();
      const body = new THREE.Mesh(new THREE.BoxGeometry(1.4, 0.5, 2.2), new THREE.MeshStandardMaterial({
color }));
      body.position.y = 0.4; body.castShadow = true; group.add(body);
      const driver = new THREE.Mesh(new THREE.SphereGeometry(0.45, 16, 16), new
THREE.MeshStandardMaterial({ color: 0xfde047 }));
      driver.position.set(0, 0.9, -0.1); group.add(driver);
      const wheelGeo = new THREE.CylinderGeometry(0.35, 0.35, 0.3, 12); wheelGeo.rotateZ(Math.PI / 2);
      const wheelMat = new THREE.MeshStandardMaterial({ color: 0x111111 });
      [[-0.8, 0.35, 0.7], [0.8, 0.35, 0.7], [-0.8, 0.35, -0.7], [0.8, 0.35, -0.7]].forEach(p => {
        const w = new THREE.Mesh(wheelGeo, wheelMat); w.position.set(...p); group.add(w);
      });
      scene.add(group); return group;
    }

    const myKart = createKart(0xff2a5f);
    const rivalKart = createKart(0x00f5d4); rivalKart.visible = false;

    const input = { left: false, right: false, gas: false, drift: false };
    let kart = { x: 0, y: 0, z: 18, vx: 0, vz: 0, speed: 0, rot: 0, vy: 0, driftCharge: 0, item: null };
    const marbles = [];

    function bindTouch(id, key) {
      const el = document.getElementById(id);
      el.addEventListener('touchstart', (e) => { e.preventDefault(); input[key] = true; if (key ===
'drift' && kart.y === 0) kart.vy = 0.22; });
      el.addEventListener('touchend', (e) => {
        e.preventDefault();
        if (key === 'drift' && kart.driftCharge > 35) kart.speed = 0.95;
```

index.html — Part 3: Main Animation & Physics Loop Closing Block — javascript

```
    function animate() {
      requestAnimationFrame(animate);
      const turnRate = input.drift ? 0.055 : 0.038;
      if (input.left) kart.rot += turnRate;
      if (input.right) kart.rot -= turnRate;

      if (input.drift && input.gas && (input.left || input.right)) {
        kart.driftCharge++;
        document.getElementById('speedDisplay').innerText = kart.driftCharge > 35 ? '🔥 ULTRA SPARK
READY!' : '⚡ CHARGING DRIFT...';
      } else {
        document.getElementById('speedDisplay').innerText = 'SPEED: ' + Math.round(kart.speed * 100) + '
KM/H';
      }

      if (input.gas) kart.speed = Math.min(kart.speed + 0.018, 0.62);
      else kart.speed *= 0.96;

      kart.x -= Math.sin(kart.rot) * kart.speed;
      kart.z -= Math.cos(kart.rot) * kart.speed;

      if (kart.x > -11 && kart.x < -5 && kart.z > -18 && kart.z < -6 && kart.speed > 0.3) kart.vy = 0.28;
      kart.y += kart.vy;
      if (kart.y > 0) kart.vy -= 0.015; else { kart.y = 0; kart.vy = 0; }

      if (Math.abs(kart.x) > 40 || Math.abs(kart.z) > 40) { kart.x = 0; kart.z = 18; kart.speed = 0; }
      obstacles.forEach(ob => { if (Math.abs(kart.x - ob.x) < ob.rx && Math.abs(kart.z - ob.z) < ob.rz)
kart.speed = -0.25; });

      itemBoxes.forEach(box => {
        box.rotation.y += 0.04; box.rotation.x += 0.02;
        if (box.visible && Math.hypot(kart.x - box.position.x, kart.z - box.position.z) < 1.8) {
          box.visible = false; setTimeout(() => box.visible = true, 5000);
          kart.item = Math.random() > 0.4 ? 'MARBLE SHELL' : 'BATTERY TURBO';
          document.getElementById('itemDisplay').innerText = 'ITEM: ' + kart.item;
        }
      });

      for (let i = marbles.length - 1; i >= 0; i--) {
        const m = marbles[i]; m.mesh.position.x += m.vx; m.mesh.position.z += m.vz; m.life--;
        if (m.life < 165 && Math.hypot(kart.x - m.mesh.position.x, kart.z - m.mesh.position.z) < 1.4) {
kart.speed = 0; kart.vy = 0.35; m.life = 0; }
        if (m.life <= 0) { scene.remove(m.mesh); marbles.splice(i, 1); }
      }

      myKart.position.set(kart.x, kart.y, kart.z);
      myKart.rotation.y = kart.rot + (input.drift ? (input.left ? 0.25 : input.right ? -0.25 : 0) : 0);
      camera.position.lerp(new THREE.Vector3(kart.x + Math.sin(kart.rot) * 7.5, kart.y + 3.8, kart.z +
Math.cos(kart.rot) * 7.5), 0.15);
      camera.lookAt(kart.x, kart.y + 0.8, kart.z);

      if (conn && conn.open) conn.send({ type: 'move', x: kart.x, y: kart.y, z: kart.z, rot: kart.rot });
      if (remoteKartData.active) {
        rivalKart.position.lerp(new THREE.Vector3(remoteKartData.x, remoteKartData.y, remoteKartData.z),
0.3);
        rivalKart.rotation.y = remoteKartData.rot;
      }
      renderer.render(scene, camera);
    }
    animate();
  </script>
</body>
</html>
```

PART 5 Auto-Build the Android .APK via GitHub Actions Cloud

.github/workflows/build-apk.yml — Automatic Capacitor Android APK Compiler — yaml

```
name: Build Desk Racer APK
on:
  push:
    branches: [ main, master ]
  workflow_dispatch:
```